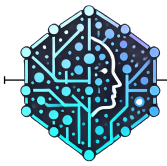


# LABORATORIO DI INTELLIGENZA ARTIFICIALE APPLICATA

## A.K.A. AI APPLICATIONS IN PRACTICE

**4 - Exceptions - Object Oriented Programming - Type Hints**

Prof. Tiberio Uricchio
< tiberio.uricchio@unipi.it >

## *EXCEPTION HANDLING*

# Introduction to Exceptions

Exceptions are events that disrupt the normal flow of a program's execution.

Exceptions are objects that represent errors. When an error occurs, Python creates an exception object, and if not handled, the program terminates with a traceback message.

```
# Example of an exception occurring
def divide(a, b):
    return a / b

# This will raise a ZeroDivisionError
result = divide(10, 0)
```



# Exception Hierarchy

Python has a rich hierarchy of built-in exceptions.

All exceptions inherit from the *BaseException* class, with the most common ones inheriting from *Exception*.

```
# Simplified exception hierarchy
# BaseException
#  ├── SystemExit                (raised by sys.exit())
#  ├── KeyboardInterrupt         (raised by Ctrl+C)
#  ├── Exception                 (base for most exceptions)
#  │    ├── ArithmeticError      (base for math errors)
#  │    │    ├── ZeroDivisionError
#  │    │    └── OverflowError
#  │    ├── LookupError          (base for key/index errors)
#  │    │    ├── KeyError
#  │    │    └── IndexError
#  │    ├── TypeError
#  │    ├── ValueError
#  │    └── ... (many more)
```

# Common Built-in Exceptions

Python includes many built-in exceptions for different error scenarios:

```
# IndexError: Accessing a non-existent sequence index
my_list = [1, 2, 3]
print(my_list[10])  # IndexError

# KeyError: Accessing a non-existent dictionary key
my_dict = {"a": 1, "b": 2}
print(my_dict["c"])  # KeyError

# TypeError: Operation with incompatible types
print("2" + 2)  # TypeError

# ValueError: Operation with correct type but incorrect value
int("abc")  # ValueError

# FileNotFoundError: Trying to access a non-existent file
open("nonexistent.txt", "r")  # FileNotFoundError
```



# Handling exceptions

The try-except construct allows you to handle exceptions gracefully:

```
try:
    # Code that might raise an exception
    num = int(input("Enter a number: "))
    result = 10 / num
    print("Result:", result)
except ZeroDivisionError:
    # Handles division by zero
    print("Cannot divide by zero!")
except ValueError:
    # Handles invalid input (e.g., letters instead of numbers)
    print("Please enter a valid number!")

# Program continues running after handling exceptions
print("Program execution continues...")
```

# Handling exceptions

If you have to handle more than one exception, you can use the following patterns:

```
# Method 1: Multiple except blocks
try:
    # Some risky operation
    data = open("data.txt").read()
    value = int(data)
except FileNotFoundError:
    print("File not found")
except ValueError:
    print("File does not contain a valid number")

# Method 2: Grouping exceptions
try:
    x = 1 / 0
except (ZeroDivisionError, TypeError):
    print("A math error occurred")
```



# Handling exceptions

Exceptions can be captured and handled based on their content:

```
# Capturing the exception instance
try:
    y = int("abc")
except ValueError as e:
    print(f"Error details: {e}")
```

Output:

```
Error details: invalid literal for int() with base 10: 'abc'
```

# Handling exceptions

Try statements can include **else** and **finally** clauses for more control:

```
try:
    file = open("data.txt", "r")
    content = file.read()
except FileNotFoundError:
    print("The file was not found")
else:
    # Runs if try block succeeds (no exceptions)
    print(f"File has {len(content)} characters")
finally:
    # Always runs, regardless of whether an exception occurred
    # Good for cleanup operations
    print("Execution completed")
    if 'file' in locals() and not file.closed:
        file.close()
        print("File closed")
```



# Raising Exceptions

You can manually raise exceptions using the *raise* statement:

```
# Raise a specific exception
def validate_age(age):
    if not isinstance(age, int):
        raise TypeError("Age must be an integer")
    if age < 0:
        raise ValueError("Age cannot be negative")
    if age > 150:
        raise ValueError("Age unrealistically high")
    return age

# Re-raising the caught exception
try:
    validate_age("twenty")
except TypeError as e:
    print(f"Error: {e}")
    raise  # Re-raises the caught exception
```

# Exception Chaining

Python supports exception chaining to preserve the original cause of an exception:

```
# Explicit exception chaining with "from"
def process_data():
    try:
        data = get_data_from_file()
        return analyze_data(data)
    except FileNotFoundError as e:
        raise ValueError("Data processing failed") from e

# Implicit chaining happens during an exception
try:
    1 / 0
except ZeroDivisionError:
    try:
        int("abc")
    except ValueError as e:
        # e.__context__ will contain the ZeroDivisionError
        print(f"Current exception: {e}")
        print(f"Caused during handling of: {e.__context__}")
```



# Best Practices for Exception Handling

Some suggestions to handle exceptions effectively:

```
# 1. Be specific: Catch only exceptions you can handle
try:
    data = json.loads(text)
except json.JSONDecodeError:  # Specific - good
    pass

# 2. Don't catch Exception or BaseException without good reason
try:
    process_data()
except Exception:  # Too broad - avoid this
    pass
```

# Best Practices for Exception Handling

```python
# 3. Keep try blocks small
try:
    value = int(input("Enter number: "))  # Only the risky code
except ValueError:
    value = 0

# 4. Use context managers for resource cleanup
with open("file.txt", "r") as file:  # file.close() happens automatically
    data = file.read()
```



# Exercises

- Write a function **calculate(op, a, b)** that:
  - Supports the operations "+", "-", "*", and “/“
  - When dividing by zero it prints "Error: cannot divide by zero.”
  - If the operator is invalid it prints "Error: unsupported operation.”
  - If a or b are not valid numbers it prints “Error: only numbers are supported.”
  - *Hint: you need to handle ZeroDivisionError, ValueError, TypeError
- Write a function **get_student_grade(student_grades, name)** that takes a dictionary *student_grades* where keys are student names and values are grades, and returns the grade of the student name.
  If the student is not in the dictionary, it prints "Error: student not found.”.

*OBJECT ORIENTED PROGRAMMING*

# Objects

We have seen several different types of objects in Python

- 42
- 3.1415
- 'Hello'
- [77, 80, 83, 99, 2, 5, 15, 17, 19]
- (4, 5, 2, 3, 6)
- {'name': 'Bob', 'age': 20}
- {'a', 'b', 'c'}

# Objects

We have seen several different types of objects in Python

- 42  INT
- 3.1415  FLOAT
- 'Hello'  STRING
- [77, 80, 83, 99, 2, 5, 15, 17, 19]  LIST
- (4, 5, 2, 3, 6)  TUPLE
- {'name': 'Bob', 'age': 20}  DICTIONARY
- {'a', 'b', 'c'}  SET

Each object has a **type**. An object is an **instance** of a type.



# Objects

Everything in Python is an Object and has a type.

Objects can be created, manipulated and destroyed.

An object can be destroyed with **del**.

When an object is deleted or when it is not accessible anymore, Python removes it from memory thanks to *the Garbage Collector*.

The Garbage Collector keeps track of how many references to an object are present, and removes unreferenced ones.

# Objects

Objects are a data abstraction with an interface for interacting with them.

Interaction with objects is made through functions.

The interface defines the behaviors and hides the implementations.



# Objects

**Behaviour**

Given a list L

- The length of a list is obtained with the function len()
- The i-th element is obtained with indexing L[i]
- ...

**Implementation**

Lists are represented internally as lined lists of cells where pointers are followed to the next index.

# Objects

**CPython Implementation**

```
typedef struct {
    PyObject_VAR_HEAD
    PyObject **ob_item;
    Py_ssize_t allocated;
} PyListObject;
```

https://github.com/python/cpython/blob/5c22476c01622f11b7745ee693f8b296a9d6a761/Include/listobject.h#L22



# Objects and Classes

Objects bundle data into packages together with functions to operate on it through an interface.

New object types can be created by defining a **Class**.

Divide and conquer

- Implement and test the functionalities of each class separately
- Increased modularity reduces complexity
- Divide the code into small and understandable parts

Defining a class allows us to reuse code easily.

# Object Oriented Programming

**Class**: defines the template for a specific object type

**Method**: a function that defines a behaviour of all the objects in the class

**Attribute**: a field with data describing the object

**Object**: a particular instance of a class



# Object Oriented Programming

**Class**: defines the template for a specific object type

**Cat**

**Method**: a function that defines a behaviour of all the objects in the class

**Attribute**: a field with data describing the object

**Object**: a particular instance of a class

# Object Oriented Programming

**Class**: defines the template for a specific object type

**Method**: a function that defines a behaviour of all the objects in the class

**Attribute**: a field with data describing the object

**Object**: a particular instance of a class

**Cat**

**meow()**



# Object Oriented Programming

**Class**: defines the template for a specific object type

**Method**: a function that defines a behaviour of all the objects in the class

**Attribute**: a field with data describing the object

**Object**: a particular instance of a class

**Cat**

**meow()**

**color**

# Object Oriented Programming

**Class**: defines the template for a specific object type

**Cat**

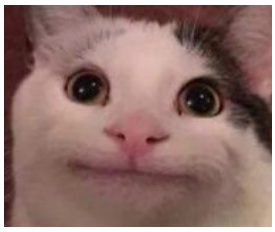
**Method**: a function that defines a behaviour of all the objects in the class

**meow()**

**Attribute**: a field with data describing the object

**color**

**Object**: a particular instance of a class



# Object Oriented Programming

**Class**: defines the template for a specific object type

**Cat**

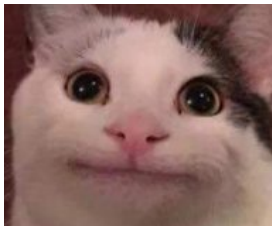
**Method**: a function that defines a behaviour of all the objects in the class

**meow()**

**Attribute**: a field with data describing the object

**color**

**Object**: a particular instance of a class



# Classes

A class defines the abstract characteristics of an object.

This includes how the object is made (attributes) and how it behaves (methods).

A class is like a blueprint used for creating the objects of a certain type.

Instances of a class share attributes (*color*, *age*, *name*) and methods (*meow()*, *jump()*, *sleep()*) with all other instances.



# Instances

Of course, the values assigned to the attributes of a class can change depending on the object instance.

The set of values assigned to the attributes of a particular object is referred to as the *state* of the object.

Behaviors instead are defined at a class level and are the same across instances.

# Methods

You can think at methods as actions that involve the object.

Typically methods have to be invoked from a specific object of the class.

If you define a class *Cat* with a method *meow()* you will need to first create an instance of a *Cat* to make it meow.



# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

## Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

A class is defined using the keyword ***class***



## Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

Class names follow the PascalCase convention

# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

Classes require a **constructor** function. This function is called when the object is created and assigns values to attributes



# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

The constructor is always defined as def __init__(self, attr1, attr2, attr3, ...)

# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

The keyword **self** indicates the object itself. It is automatically passed to class methods and you can use it to operate on object attributes or call other methods.



# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

You can pass a list of attributes to the constructor when you create the object

## Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

With **self.attr_name** you can refer to attributes



## Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

You can define methods in the class declaration as normal functions. To call this function you will need to call obj.meow() where obj is an object of the class Cat

## Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

You can create an object of class *Cat* by calling its constructor and passing the expected arguments



## Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
mycat.meow()


>> Meow!
```

You can call the methods of the class with object.method()

# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name):
        self.name = name

    def meow(self):
        print('Meow!')

mycat = Cat('bob')
print(type(mycat))


>> <class '__main__.Cat'>
```

The class of an object can be checked with *type*, as you would do for regular built-in objects.



# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.is_hungry = False

    def meow(self):
        print('{} says: Meow!'.format(self.name))

    def get_color(self):
        print(self.color)
        return self.color

    def eat(self):
        self.is_hungry = False

bob = Cat('Bob', 'red')

print(dir(bob))


>> ['__class__', '__delattr__', '__dict__', '__dir__',
'__doc__', '__eq__', '__format__', '__ge__', '__getattribute__',
'__gt__', '__hash__', '__init__', '__init_subclass__', '__le__',
'__lt__', '__module__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__str__', '__subclasshook__', '__weakref__', 'color', 'eat',
'get_color', 'is_hungry', 'meow', 'name']
```

The function ***dir()*** can be used to check all the attributes and methods of an object.

Methods starting and ending with a double underscore _ _ are special methods that are invoked by a particular syntax.

Ad example __init__() is called when the object is created.

# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.is_hungry = False

    def meow(self):
        print('{} says: Me

    def get_color(self):
        print(self.color)
        return self.color

    def eat(self):
        self.is_hungry = F

bob = Cat('Bob', 'red')

print(dir(bob))


>> ['__class__', '__delat
'__doc__', '__eq__', '__f
'__gt__', '__hash__', '__
'__lt__', '__module__', '__ne__', '__new__', '__reduce__',
'__reduce_ex__', '__repr__', '__setattr__', '__sizeof__',
'__str__', '__subclasshook__', '__weakref__', 'color', 'eat',
'get_color', 'is_hungry', 'meow', 'name']
```

What happens if you use ***dir()*** on built-in types?

```
>>> dir('abcde')
['__add__', '__class__', '__contains__', '__del
attr__', '__dir__', '__doc__', '__eq__', '__for
mat__', '__ge__', '__getattribute__', '__getite
m__', '__getnewargs__', '__gt__', '__hash__', '
__init__', '__init_subclass__', '__iter__', '__
le__', '__len__', '__lt__', '__mod__', '__mul__
', '__ne__', '__new__', '__reduce__', '__reduce
_ex__', '__repr__', '__rmod__', '__rmul__', '__
setattr__', '__sizeof__', '__str__', '__subclas
shook__', 'capitalize', 'casefold', 'center', '
count', 'encode', 'endswith', 'expandtabs', 'fi
nd', 'format', 'format_map', 'index', 'isalnum'
, 'isalpha', 'isascii', 'isdecimal', 'isdigit',
'isidentifier', 'islower', 'isnumeric', 'isprin
table', 'isspace', 'istitle', 'isupper', 'join'
, 'ljust', 'lower', 'lstrip', 'maketrans', 'par
tition', 'replace', 'rfind', 'rindex', 'rjust',
'rpartition', 'rsplit', 'rstrip', 'split', 'spl
itlines', 'startswith', 'strip', 'swapcase', 't
itle', 'translate', 'upper', 'zfill']
```

The function ***dir()*** can be used to check all the attributes and methods of an object.

Methods starting and ending with a double underscore _ _ are special methods that are invoked by a particular syntax.

And example __init__() is called when the object is created.



# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.is_hungry = False

    def meow(self):
        print('{} says: Meow!'.format(self.name))

    def get_color(self):
        print(self.color)
        return self.color

    def eat(self):
        self.is_hungry = False

bob = Cat('Bob', 'red')
kit = Cat('Kit', 'white')

bob.meow()
kit.meow()
```

Different objects have different behaviors, depending on their attributes.

# Object Oriented Programming in practice

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color
        self.is_hungry = False

    def meow(self):
        print('{} says: Meow!'.format(self.name))

    def get_color(self):
        print(self.color)
        return self.color

    def eat(self):
        self.is_hungry = False

bob = Cat('Bob', 'red')

print(bob.is_hungry)
bob.is_hungry = True
print(bob.is_hungry)
bob.eat()
print(bob.is_hungry)

>> False
   True
   False
```

Attributes can be changed by accessing them with the **obj.attr**

Attributes can also be changed by methods of the object.



# PEP8 for classes

The following recommendations are conventional:

- **`joined_lower`** for functions, methods, attributes, variables
- **`joined_lower`** or ALL_CAPS for constants
- **`StudlyCaps (PascalCase)`** for classes
- **`camelCase`** only to conform to pre-existing conventions

Unfortunately, the naming conventions of Python's library are a bit of a mess, so it will be hard to have it completely consistent

# Overriding

```
class Cat:
    def __init__(self, name, color):
        self.name = name
        self.color = color

    def __repr__(self):
        return 'A nice {} Cat called {}'.format(self.color,
                                                self.name)

    def meow(self):
        print('{} says: Meow!'.format(self.name))

bob = Cat('Bob', 'red')
kit = Cat('Kit', 'white')

print(bob)
print(kit)


>> A nice red Cat called Bob
   A nice white Cat called Kit
```

***__repr__()*** is a function that returns a string describing an object. It is invoked when you call **print()**.

The ***__repr__()*** function can be explicitly redefined to exhibit a custom behavior.

The act of redefining an existing function is known as **OVERRIDING.**



# Public vs. Private

All attributes we have defined since here can be accessed and modified from everywhere in the code.

```
class Cat:
    def __init__(self, name, color):
        # We only want strings as names!
        assert(type(name) == str)
        self.name = name

bob = Cat('Bob', 'red')
print(bob.name)
bob.name = 1992
print(bob.name)

>> Bob
   1992
```

This may lead to some unwanted behavior, such as setting a name as an integer.

When attributes are accessible everywhere, they are called ***public.***

# Public vs. Private

You may want to «protect» the attributes of a class by making them ***private***. This means that they can be accessed or modified only from inside the function itself.

When an attribute is private you need to define functions to interact with them:

A **getter** function is used to retrieve the value of a certain field.

A **setter** function is used to change the value of a certain field.



# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        self.public_name = name
        self.__private_name = name
        self.color = color

bob = Cat('Bob', 'red')
print(bob.public_name)
print(bob.__private_name)



>> Bob
Traceback (most recent call last):
  File "public_private.py", line 19, in <module>
  print(bob.__private_name)
AttributeError: 'Cat' object has no attribute '__private_name
```

# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        self.public_name = name
        self.__private_name = name
        self.color = color

bob = Cat('Bob', 'red')
print(bob.public_name)
print(bob.__private_name)



>> Bob
Traceback (most recent call last):
  File "public_private.py", line 19, in <module>
  print(bob.__private_name)
AttributeError: 'Cat' object has no attribute '__private_name
```

A private attribute is specified with a double underscore __ preceding its name



# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        self.public_name = name
        self.__private_name = name
        self.color = color

bob = Cat('Bob', 'red')
print(bob.public_name)
print(bob.__private_name)



>> Bob
Traceback (most recent call last):
  File "public_private.py", line 19, in <module>
  print(bob.__private_name)
AttributeError: 'Cat' object has no attribute '__private_name
```

Private members are not accessible from outside the class

# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        assert(type(name) == str)
        self.__name = name
        self.color = color

    def __repr__(self):
        return 'A nice {} Cat called {}'.format(self.color,
                                                self.__name)

    def change_name(self, name):
        self.__name = name

    def print_name(self):
        print(self.__name)

mycat = Cat('Snowball', 'black')
mycat.print_name()
mycat.change_name('Snowball II')
mycat.print_name()


>> Snowball
   Snowball II
```



# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        assert(type(name) == str)
        self.__name = name
        self.color = color

    def __repr__(self):
        return 'A nice {} Cat called {}'.format(self.color,
                                                self.__name)

    def change_name(self, name):
        self.__name = name

    def print_name(self):
        print(self.__name)

mycat = Cat('Snowball', 'black')
mycat.print_name()
mycat.change_name('Snowball II')
mycat.print_name()


>> Snowball
   Snowball II
```

A private variable can only be changed from a method of the class. This is a **setter** function.

# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        assert(type(name) == str)
        self.__name = name
        self.color = color

    def __repr__(self):
        return 'A nice {} Cat called {}'.format(self.color,
                                                self.__name)

    def change_name(self, name):
        self.__name = name

    def print_name(self):
        print(self.__name)

mycat = Cat('Snowball', 'black')
mycat.print_name()
mycat.change_name('Snowball II')
mycat.print_name()


>> Snowball
   Snowball II
```

A private variable can only be retrieved from a method of the class. This is a **getter** function.



# Public vs. Private

```
class Cat:
    def __init__(self, name, color):
        assert(type(name) == str)
        self.__name = name  # private
        self._color = color # semi-private

    def __repr__(self):
        return 'A nice {} Cat called {}'.format(self.color,
                                                self.__name)

    def change_name(self, name):
        self.__name = name

    def print_name(self):
        print(self.__name)

mycat = Cat('Snowball', 'black')
print(mycat._color)


>> black
```

It works, but...

Attributes that start with only one underscore _ are considered «semi-private». These are still public but the programmer has marked them as if they were private.

The code will work if you do, but they should not be used outside from the class.

# Scopes and Namespaces

```
def scope_test():
    def do_local():
        spam = "local spam"

    def do_nonlocal():
        nonlocal spam
        spam = "nonlocal spam"

    def do_global():
        global spam
        spam = "global spam"

    spam = "test spam"
    do_local()
    print("After local assignment:", spam)
    do_nonlocal()
    print("After nonlocal assignment:", spam)
    do_global()
    print("After global assignment:", spam)

scope_test()
print("In global scope:", spam)
```

Output

```
After local assignment: test spam
After nonlocal assignment: nonlocal spam
After global assignment: nonlocal spam
In global scope: global spam
```

If a name is declared **global**, then all references and assignments go directly to the next-to-last scope containing the module's global names.

The **nonlocal** keyword rebinds variables found outside of the innermost scope.

if
no **global** or **nonlocal** statement is in effect, assignments to names always go into the **innermost scope**.



# Encapsulation

Object Oriented Programming (OOP) offers the advantage of **encapsulating** data inside classes.

This is often used to hide internal representations only exposing an interface for interaction→ **information hiding**

If you have an attribute that is not visible from the outside of an object, and pair it with methods that provide read or write access to it, then you can hide specific information and control access to the internal state of the object.

# Operator Overloading

```
class Cat:
    def __init__(self, name, sex, p1=None, p2=None):
        self.__name = name
        self.__sex = sex
        self.__parent1 = p1
        self.__parent2 = p2

    def change_name(self, name):
        self.__name = name

    def get_name(self):
        return self.__name

    def __add__(self, other_cat):
        print('A kitten is born!')
        if random() > 0.5:
            s = 'male'
        else:
            s = 'female'
        return Cat(name='', sex=s, p1=self, p2=other_cat)

    def __repr__(self):
        return 'A {} Cat called {} born from {} and {}'.format(
                  self.__sex, self.__name,
                  self.__parent1.get_name(), self.__parent2.get_name())

cat1 = Cat('Mr Purr', 'male', 'black')
cat2 = Cat('Mrs Purr', 'female', 'red')

cat3 = cat1 + cat2
cat3.change_name('Purr Jr.')
print(cat3)


>> A kitten is born!
   A female Cat called Purr Jr. born from Mr Purr and Mrs Purr
```

Standard operators (+, -, *, ecc.) can be **overloaded** to define how objects of your class should behave.



# Operator Overloading

| OPERATOR | |
|---|---|
| + | __add__(self, other) |
| – | __sub__(self, other) |
| * | __mul__(self, other) |
| / | __truediv__(self, other) |
| // | __floordiv__(self, other) |
| % | __mod__(self, other) |
| ** | __pow__(self, other) |

# Operator Overloading

| OPERATOR | |
|---|---|
| < | __lt__(self, other) |
| > | __gt__(self, other) |
| <= | __le__(self, other) |
| >= | __ge__(self, other) |
| == | __eq__(self, other) |
| != | __ne__(self, other) |



# Operator Overloading

| OPERATOR | |
|---|---|
| -= | __isub__(self, other) |
| += | __iadd__(self, other) |
| *= | __imul__(self, other) |
| /= | __idiv__(self, other) |
| //= | __ifloordiv__(self, other) |
| %= | __imod__(self, other) |
| **= | __ipow__(self, other) |

# Inheritance

An existing class can be reused and make another class ***inherit*** its characteristics.

The new class will have all the capabilities of the old class, and can be modified in order to have some different behavior.

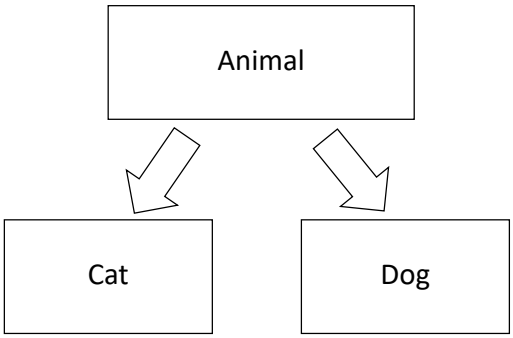
The two classes are referred to as *Parent* and *Children*.

A child class is also known as **subclass**, and is a specialized version of a **superclass**.



# Inheritance

```
class Animal:
    def __init__(self, name, color):
        self.name = name
        self.color = color

    def make_sound(self):
        pass

class Cat(Animal):
    def make_sound(self):
        print('meow')

class Dog(Animal):
    def make_sound(self):
        print('bau')

garfield = Cat(name='Garfield', color='orange')
lassie = Dog(name='Lassie', color='brown')

garfield.make_sound()
lassie.make_sound()


>> meow
   bau
```




Cat and Dog are both animals and they share some common characteristics and behaviours.

In this example both the classes Cat and Dog inherit from the superclass Animal.

# Inheritance

```
class Person:

    def __init__(self, person_name, person_age):
        self.name = person_name
        self.age = person_age

    def show_name(self):
        print(self.name)

    def show_age(self):
        print(self.age)


class Student(Person):

    def __init__(self, student_name, student_age, student_id):
        super().__init__(student_name, student_age)
        self.student_id = student_id

    def get_id(self):
        return self.student_id

pers = Person("Richard", 23)
pers.show_age()
stud = Student("Max", 22, "102")
print(stud.get_id())
stud.show_name()


>> 23
   102
   Max
```

If we want to add attributes to the subclass we can override the __init__ function.

The __init__ function of the parent class can still be invoked by calling super().__init__()

With **super()** you can access methods of the parent class from a children



# Inheritance built-in functions

Use **`isinstance()`** to check an **instance**'s type:

```
isinstance(obj, int)
```

will be **True** only if **`obj.__class__`** is **int** or **some class derived from int**.

Use **`issubclass()`** to check **class** inheritance:

```
issubclass(bool, int)
```

is **True** since **bool** is a **subclass of int**.

However, **`issubclass(float, int)`** is **False** since **float** is not a subclass of **int**.

# Multiple Inheritance

Python supports a form of multiple inheritance. When any function or attribute is used, you can think of a search from the derived class to each of the parent classes.

```
class DerivedClassName(Base1, Base2, Base3):
    <statement-1>
    .
    .
    .
    <statement-N>
```

If an attribute is not found in **DerivedClassName**, it is searched for in Base1, then (recursively) in the base classes of Base1. If it is not found there, it will be searched in Base2, and so on.



# Polymorphism

**Polymorphism** is the ability to leverage the same interface for different data types or classes.

With polymorphism a function can be used regardless of the class of the object it operates on.

When several classes or subclasses have the same method names but different implementations then the classes are polymorphic.

Polymorphism can be carried out with inheritance, when subclasses make use of methods of the parent class or when they override them.

# Polymorphism

```
class Shark():
    def swim(self):
        print("The shark is swimming.")

    def swim_backwards(self):
        print("The shark cannot swim backwards, but can sink backwards.")

    def skeleton(self):
        print("The shark's skeleton is made of cartilage.")


class Clownfish():
    def swim(self):
        print("The clownfish is swimming.")

    def swim_backwards(self):
        print("The clownfish can swim backwards.")

    def skeleton(self):
        print("The clownfish's skeleton is made of bone.")
```



# Creating Custom Exceptions

You can define your own exception classes by inheriting from *Exception*:

```
# Custom exception class
class NetworkError(Exception):
    """Exception raised for network-related errors."""
    pass

class DatabaseError(Exception):
    """Exception raised for database-related errors."""
    def __init__(self, message, error_code):
        self.message = message
        self.error_code = error_code
        super().__init__(f"Error {error_code}: {message}")

# Using custom exceptions
try:
    raise DatabaseError("Connection timeout", 4004)
except DatabaseError as e:
    print(f"Database issue: {e}")
    print(f"Error code: {e.error_code}")
```

# Best Practices for Exception Handling

This add another common practice on how to handle errors:

```
# 5. Use exception hierarchy to your advantage
class AppError(Exception): pass
class ConfigError(AppError): pass
class NetworkError(AppError): pass

# Can catch all app errors or specific types
try:
    setup_app()
except ConfigError:
    # Handle configuration errors
except AppError:
    # Handle all other app errors
```



# Exercises

- Define a **TypedList class** inheriting from the built-in class list. A type must be defined at creation. Override the append method so that only objects of the same type can be appended.
- Define a **class Patient** and a **class Medicine**. Each patient consists of a name, age and a set containing what substances he/she is allergic to. The class medicine instead consist of a set of allergens. Within the class Patient, define a method to check whether the patient is allergic to a medicine i.e. any of its allergens.

*PYTHON TYPE HINTS*

# Introduction to Type Hints

Python type hints (introduced in PEP 484) provide optional static typing.

They allow to specify the expected types of variables, function parameters, and return values.

Type hints don't affect runtime behavior but enable:

- Better code documentation
- Enhanced IDE support (auto-completion, error detection)
- Static type checking with tools like mypy
- Improved code maintainability

# Introduction to Type Hints

```python
# Without type hints
def greeting(name):
    return "Hello, " + name

# With type hints
def greeting(name: str) -> str:
    return "Hello, " + name
```



# Basic Type Annotations

Python offers several basic types for annotations:

- `int`: Integer values
- `float`: Floating-point numbers
- `bool`: Boolean values (True/False)
- `str`: String values
- `bytes`: Byte sequences
- `None`: The None type

```python
age: int = 25
height: float = 1.85
is_student: bool = True
name: str = "Alice"
data: bytes = b"binary data"
nothing: None = None
```

# Function Type Hints

Function type hints specify the types of parameters and return values:

```python
def calculate_area(length: float, width: float) -> float:
    return length * width

def is_adult(age: int) -> bool:
    return age >= 18

def greet(name: str) -> None:
    print(f"Hello, {name}!")
    # No return value needed when returning None
```



# Complex Data Types

For built-in types, the typename can be used directly (from Python 3.10):

```python
# Lists
numbers: list[int] = [1, 2, 3, 4, 5]

# Dictionaries
user_info: dict[str, str] = {"name": "Alice", "email": "alice@example.com"}

# Tuples
coordinates: tuple[float, float] = (10.5, 20.3)

# Fixed-size tuples with different types
record: tuple[str, int, float] = ("Alice", 30, 1.75)

# Sets
unique_ids: set[int] = {1, 2, 3, 4, 5}
```

# Complex Data Types

Before Python 3.9 (PEP 585), standard library generic types like **list** could not be parameterized at runtime (i.e., **list[int]** would throw an error).

In that case, complex types can be imported from the **`typing`** module:

```python
from typing import List, Dict, Tuple, Set

# Lists
numbers: List[int] = [1, 2, 3, 4, 5]

# Dictionaries
user_info: Dict[str, str] = {"name": "Alice", "email": "alice@example.com"}

# Tuples
coordinates: Tuple[float, float] = (10.5, 20.3)

# Fixed-size tuples with different types
record: Tuple[str, int, float] = ("Alice", 30, 1.75)

# Sets
unique_ids: Set[int] = {1, 2, 3, 4, 5}
```



# Union Types

Union types indicate that a value can be one of several types:

```python
from typing import Union, Optional

# Value can be either int or str
user_id: Union[int, str] = 123

# Later in code
user_id = "ABC123"  # Still valid

# Optional is a shorthand for Union[Type, None]
# Indicates a value that might be None
result: Optional[str] = get_result()
# Equivalent to: result: Union[str, None] = get_result()
```

# Union Types

The use of Optional is now deprecated. Starting from Python 3.12 you can use the following syntax:

- `int | str`
  is the same as `Union[int, str]`
- `int | str | range`
  is the same as `Union[int, str, range]`
- `int | None`
  is the same as `Optional[int]` and `Union[int, None]`

```python
# Python 3.10+ Union type syntax
# Old way:
from typing import Union
value: Union[int, str] = 42

# New way:
value: int | str = 42
```



# Type Aliases

Type aliases create custom named types for complex type annotations (introduced in PEP 613, Python 3.10):

```python
Url = str

def retry(url: Url, retry_count: int) -> None: ...
```

or by using typing.TypeAlias:

```python
from typing import TypeAlias

Url: TypeAlias = str

def retry(url: Url, retry_count: int) -> None: ...
```

# Type Aliases

Complex types can be aliased by importing the relevant classes:

```
from typing import Dict, List, Tuple, Union, TypeAlias

# Simple alias
UserId = int

# Complex aliases
Coordinates = Tuple[float, float]
Point2D = Dict[str, float]  # {"x": 1.0, "y": 2.0}

# Even more complex
JSON = Union[Dict[str, "JSON"], List["JSON"], str, int, float, bool, None]

# Python 3.10+ syntax
Temperature: TypeAlias = float
```



# match-case statement

From Python 3.10, it is possible to directly react based on types:

```
# Python 3.10+ - Match patterns with type guards
def process_data(data: int | str | list):
    match data:
        case int():
            return data + 1
        case str():
            return data.upper()
        case list():
            return sum(data)
```

# Callable types

The Callable type hint is used for functions and other callable objects:

```
from typing import Callable

# Function that takes two ints and returns a float
OperationFunc = Callable[[int, int], float]

def apply_operation(x: int, y: int, operation: OperationFunc) -> float:
    return operation(x, y)

def divide(a: int, b: int) -> float:
    return a / b

# Usage
result = apply_operation(10, 5, divide)  # 2.0
```



# Generic types

Generic types allow you to define classes and functions that work with any type:

```
from typing import TypeVar, Generic, List

T = TypeVar('T')  # Declare a type variable

class Stack(Generic[T]):
    def __init__(self) -> None:
        self.items: List[T] = []

    def push(self, item: T) -> None:
        self.items.append(item)

    def pop(self) -> T:
        return self.items.pop()

# Usage
int_stack = Stack[int]()  # Stack of integers
str_stack = Stack[str]()  # Stack of strings
```

# Exercises

- Below is a function that calculates the area of a rectangle. Add appropriate type hints to indicate the expected input and return types.

```
def rectangle_area(length, width):
    return length * width
```

- Complete the following function by adding the correct type hints. The function takes a list of numbers and returns a dictionary containing the sum and average of the numbers.

```
def analyze_numbers(numbers):
    total = sum(numbers)
    average = total / len(numbers) if numbers else 0
    return {"sum": total, "average": average}
```

- Complete the exercise on **Patient** and **Medicine** classes, by adding the correct type hints.